

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ПРОЕКТУ

3.1. Технологічний стек

Для реалізації вебдодатку обрано наступний технологічний стек:

[Таблиця 3.1 — Технологічний стек системи NetVigil \(формат Excel/XLSX\)](#)

3.2. Реалізація серверної частини (Flask API)

3.2.1. Структура проекту

```
web-app/
├─ app.py # Основний Flask-додаток
├─ requirements.txt # Залежності Python
├─ Dockerfile # Збірка контейнера
├─ static/
├─ index.html # SPA-інтерфейс
├─ css/
│   └─ style.css # Стили
├─ js/
└─ app.js # Логіка SPA
```

3.2.2. REST API Endpoints

[Таблиця 3.2 — API Endpoints вебдодатку NetVigil \(формат Excel/XLSX\)](#)

3.2.3. Інтеграція з Loki

Для отримання логів використовується Loki HTTP API з LogQL-запитами:

```
def _loki_query(query: str, limit: int = 100, start=None, end=None):
    params = {"query": query, "limit": limit, "direction": "backward"}
    if start:
        params["start"] = start
    if end:
        params["end"] = end
    resp = httpx.get(f"{LOKI_URL}/loki/api/v1/query_range",
        params=params, timeout=10.0)
    if resp.status_code == 200:
        return resp.json().get("data", {}).get("result", [])
```

```
return []
```

LogQL-запити дозволяють фільтрувати логи за контейнером та ключовими словами: - `{container="nginx-proxy"}` — логи конкретного контейнера; - `{container=~"wordpress-app|nginx-proxy"} |~ "(?i)error"` — пошук за ключовим словом.

3.2.4. Інтеграція з Prometheus

Метрики системи отримуються через PromQL-запити:

```
def _prometheus_query(query: str):  
    resp = httpx.get(f"{PROMETHEUS_URL}/api/v1/query",  
        params={"query": query}, timeout=10.0)  
    if resp.status_code == 200:  
        return resp.json().get("data", {}).get("result", [])  
    return []
```

Основні PromQL-запити: - CPU: `100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)` - Memory: `(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100` - HTTP Rate: `sum(rate(nginx_http_requests_total[5m]))`

3.3. Реалізація клієнтської частини (SPA Dashboard)

3.3.1. SPA-архітектура

Клієнтська частина реалізована як Single Page Application з hash-based маршрутизацією. Навігація між сторінками виконується без перезавантаження сторінки:

```
function navigateTo(page) {  
    document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));  
    document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));  
    document.querySelector(`#page-${page}`).classList.add('active');  
    document.querySelector(`[data-page="${page}"]`).classList.add('active');  
}
```

3.3.2. Візуалізація даних (Chart.js)

Для побудови графіків використовується бібліотека Chart.js. Графік Timeline відображає кількість HTTP-запитів та помилок за останні 6 годин:

```
chartTimeline = new Chart(ctx, {
  type: 'line',
  data: {
    labels: data.requests.map(d => d.time),
    datasets: [
      { label: 'Requests/s', data: data.requests.map(d => d.value),
        borderColor: '#3b82f6', fill: true, tension: 0.4 },
      { label: 'Errors/s', data: data.errors.map(d => d.value),
        borderColor: '#ef4444', fill: true, tension: 0.4 }
    ]
  }
});
```

3.3.3. Дизайн-система

CSS побудований на змінних (CSS Custom Properties) для забезпечення консистентності:

```
:root {
  --bg-primary: #0a0e17;
  --bg-card: rgba(17, 24, 39, 0.8);
  --accent-blue: #3b82f6;
  --accent-red: #ef4444;
  --accent-green: #10b981;
  --gradient-brand: linear-gradient(135deg, #3b82f6, #06b6d4);
}
```

3.4. Інтеграція з LLM (Ollama/Llama-3)

3.4.1. Prompt Engineering

Ключовим елементом інтеграції є формування ефективних prompt-шаблонів для LLM. Промпт для аналізу логів включає:

1. **Роль** — визначення контексту («You are a cybersecurity expert»);
2. **Дані** — текст логів або опис трафіку;
3. **Формат відповіді** — JSON-схема з полями risk_score, threats, summary;
4. **Категорії** — перелік 10 типів загроз для класифікації.

3.4.2. API взаємодії з Ollama

```
def _ollama_generate(prompt: str, format_json: bool = True):
    payload = {
        "model": LLM_MODEL,
        "prompt": prompt,
        "stream": False,
    }
    if format_json:
        payload["format"] = "json"
    resp = httpx.post(f"{OLLAMA_URL}/api/generate",
        json=payload, timeout=120.0)
    if resp.status_code == 200:
        raw = resp.json().get("response", "")
        return json.loads(raw) if format_json else {"text": raw}
    return None
```

3.5. Модуль аналізу PCAP-файлів

3.5.1. Парсер PCAP

Реалізовано власний парсер формату libpcap без зовнішніх залежностей. Парсер обробляє:

- **Global Header** (24 байти) — magic number, версія, snaplen, тип мережі;
- **Packet Header** (16 байт) — timestamp, довжина пакету;
- **Ethernet Header** (14 байт) — MAC-адреси, тип протоколу;
- **IP Header** — адреси джерела та призначення, протокол;
- **TCP/UDP Header** — порти, визначення відомих сервісів.

Парсер автоматично визначає протоколи: HTTP, HTTPS/TLS, SSH, MySQL, DNS, ICMP, ARP, IPv6.

3.5.2. Потік аналізу PCAP

1. Користувач завантажує файл через drag-and-drop або кнопку;
2. Серверна частина валідує формат та розмір (макс. 10 МБ);
3. Парсер витягує до 500 пакетів з базовою інформацією;
4. Формується текстовий опис трафіку (статистика + перші 50 пакетів);
5. Опис відправляється до LLM для аналізу загроз;
6. Результати повертаються у JSON-форматі та візуалізуються на Dashboard.

3.5.3. Оптимізація продуктивності та сумісності парсера

У процесі тестування було виявлено та усунено низку проблем, пов'язаних із сумісністю різних форматів файлів трафіку та продуктивністю вебсервера під час виконання тривалих запитів до LLM:

1. Сумісність форматів часу та лінк-рівня:

- Додано підтримку часових міток наносекундної точності (ідентифікується за magic numbers 0xA1B23C4D та 0x4D3CB2A1).
- Реалізовано розбір заголовків типу Linux Cooked Capture (LINKTYPE_LINUX_SLL, ID 113), що часто створюється утилітою tcpdump при зніманні трафіку з усіх інтерфейсів (-i any), а також Raw IP (ID 101).
- Впроваджено стійкість до помилок переповнення типу OverflowError та ValueError при конвертації часових міток у пошкоджених або неповних пакетах.

Додано перевірку на формат PCAPNG (0x0A0D0D0A) з виведенням зрозумілої інструкції користувачу для попереднього конвертування у сумісний формат PCAP.

Оптимізація моделі обробки запитів вебсервера:

- Стандартні синхронні воркери Gunicorn (sync) замінено на багатопотокові (gthread із конфігурацією --threads 4). Це запобігає повному блокуванню вебсервера при тривалих операціях очікування відповідей від локальної моделі Ollama.
- Таймаут роботи воркера збільшено з 120 до 300 секунд, що забезпечує стабільне завершення тривалого аналізу трафіку без виникнення клієнтських помилок з'єднання (NetworkError).

3.6. CI/CD Pipeline (Jenkins)

Jenkinsfile автоматизує повний цикл розгортання:

```
pipeline {
  agent any
  stages {
    stage('1. Підготовка') { /* Checkout, credentials, networks */ }
    stage('2. Очистка') { /* docker compose down */ }
    stage('3. Full Deploy') {
      steps {
        sh """
        cd app-infrastructure
```

```
docker compose -f docker-compose.apps.yml up -d
docker compose -f docker-compose.monitoring.yml build --no-cache
docker compose -f docker-compose.monitoring.yml up -d
"""
}
}
stage('4. Generate PDFs') { /* Документація */ }
stage('5. Health Checks') {
/* Перевірка: nginx-proxy, mysql-db, prometheus,
telegram-bot, mysql-exporter, ai-adapter, web-app */
}
}
}
```

3.7. Метрики та моніторинг системи (Prometheus & Loki)

Health Check включає перевірку працездатності вебдодатку NetVigil у списку контейнерів.